Assignment – 3

Parallel Generation of the Mandelbrot Set

SE3082 – Parallel Computing

3rd year 1st semester - 2026

IT23575608 – H. W. Ranwala

Contents

1. Parallelization Strategies

Generating the Mandelbrot set is a heavy, compute-bound task based on a simple rule: we run a math loop on every pixel until it escapes a set boundary or hits our maximum limit of 1000 iterations.

$$z_{n+1} = z_n^2 + c$$

The main problem is that the workload is completely uneven across the image:

- **The Center**: Pixels inside the main fractal shapes never escape, so they always run for the absolute maximum 1000 iterations.
- **The Background**: Pixels outside the fractal escape almost instantly after just 1 or 2 loops.

This creates a massive load imbalance. If you split the image evenly, some processing units will finish instantly while others get stuck working forever. To fix this, I designed and implemented three different parallel approaches: OpenMP (shared memory), MPI (distributed message passing), and CUDA (GPU acceleration).

1.1 OpenMP Strategy (Shared Memory Architecture)

For the shared-memory version, I used loop-level parallelism on the outer row loop (the Y-axis). Spawning threads here instead of on the inner pixel loop gives each thread a larger chunk of work, which keeps thread management overhead low.

- **Load Balancing:** To resolve the irregular workload, I used dynamic loop scheduling: *#pragma omp parallel for schedule(dynamic, 16).* Instead of a static block layout that leaves threads idle while others struggle with the heavy center rows, dynamic scheduling treats the image as a task pool. Threads grab 16 rows at a time, keeping the CPU cores fully utilized.
- **Race Prevention & Scoping**: Variables like coordinates and counters (x, cr, ci, zr, zi, iter) are declared inside the parallel loop body, making them implicitly thread private. The output array is a shared flat buffer, but because each thread writes to a unique index offset (y * width + x), execution is completely lock-free.

$$\text{Index} = y \times \text{Width} + x$$

threads write directly to their designated destinations without synchronization primitives, avoiding cache line bouncing or lock contention.

1.2 MPI Strategy (Distributed Memory Architecture)

Because MPI processes do not share memory, I used a centralized **Master-Worker design** to handle the irregular workload across different processes.

- **Master-Worker Architecture**: Process Rank 0 acts purely as the Master coordinator, while Ranks 1 through N-1 are Workers. The Master does no math; it tracks row indices, collects results, and handles file I/O.
- **Dynamic Load Balancing**: Workers use a pull-based messaging protocol to request a row index from the Master on-demand. This ensures faster processes computing background space grab more work, while workers bogged down by the cardioid are not penalized by static overallocation.
- **Communication Optimization**: To minimize message-passing latency, workers pack data into a single 1D array of size width + 1. The first slot (buffer[0]) holds the row index metadata, and the rest hold the row's iteration counts. This allows returning an entire computed row in a single MPI_Send call, which the Master unpacks using a direct memcpy.

$$\text{Buffer Size} = \text{Width} + 1$$

The very first element in this array is reserved for the row number (the metadata), and the rest of the slots hold the actual pixel iteration counts. The worker transmits this entire block back to the Master in a single MPI_Send call. Packing the data this way optimizes our throughput because we only pay the network latency penalty once per row. It also makes things incredibly easy for the Master, which can just unpack the message and drop the data straight into the main image array using a fast memcpy.

1.3 CUDA Strategy (Massively Parallel SIMT Architecture)

The CUDA version changes our approach from row-by-row chunks to a massively parallel thread mapping on the GPU.

- **Fine-Grained Thread Mapping**: Instead of assigning entire rows to a thread, I mapped one individual thread to one single pixel (x, y). For a 4000 x 3000 image, this spawns 12,000,000 independent threads, completely filling the GPU's Streaming Multiprocessors (SMs) and letting the hardware hide memory latency by swapping active threads instantly.
- **Grid Layout:** Threads are grouped into 2D blocks. Testing showed that 16x16 blocks (256 threads) optimized hardware residency and aligned perfectly with the 32-thread hardware warp size.
- **Warp Divergence**: Threads in a warp execute in lockstep. Branch divergence occurs if adjacent threads hit different iteration paths, stalling execution. Because the Mandelbrot set features high spatial coherence, neighboring pixels escape at similar steps. A compact 2D block layout ensures warp threads map to adjacent coordinates, minimizing divergence.

2. Runtime Configurations

2.1 Hardware Specifications

The computational testing was divided into two distinct environments to accommodate the different parallelization paradigms: a local server for the CPU-bound OpenMP and MPI implementations, and a cloud-based environment for the GPU-bound CUDA implementation.

Local CPU Environment (OpenMP & MPI)

- **Host System**: Local Ubuntu Server (Hardware: Acer V5-131)
- **Central Processing Unit (CPU)**: Intel Celeron 1007U (Ivy Bridge Architecture, 2 Physical Cores, 2 Logical Threads, Base Clock 1.50 GHz, L3 Cache 2 MB).
- **Random Access Memory (RAM)**: 8 GB DDR3 Dual-Channel @ 1333 MT/s (reported as 2x 4GB modules running at a 666 MHz base clock).
- **Local Storage**: 500 GB HDD (Seagate ST500LT012, 5400 RPM, SATA 3.0).

Cloud GPU Environment (CUDA)

- **Host Platform**: Google Colaboratory (Colab)
- **Graphics Processing Unit (GPU)**: NVIDIA Tesla T4 (Turing Architecture, 40 Streaming Multiprocessors, 2560 CUDA Cores, Boost Clock up to 1590 MHz, 16 GB GDDR6 VRAM, PCIe Gen3 x16 interface).

2.2 Software and Compilation Environment

- **Operating System**: Ubuntu 22.04.3 LTS (Linux Kernel 6.2.0, 64-bit).
- **C/C++ Compiler**: GCC version 11.4.0 (for Serial and OpenMP).
- **MPI Library**: OpenMPI version 4.1.2.
- **CUDA Toolchain**: NVIDIA CUDA Compiler (NVCC) version 12.1.

2.3 Configuration Parameters Checked

- **Unified Workload**: 4000 x 3000 resolution, 1000 iteration limit.
- **OpenMP sweep**: Tested threads: 1, 2, 4, 8, 16.
- **MPI sweep**: Tested processes: 1, 2, 4, 8, 16 (with --oversubscribe flag enabled for multi-process testing on limited cores).
- **CUDA Block configurations**: Block sizes: 8x8 (64 threads), 16x16 (256 threads), 32x32 (1024 threads), 16x8 (128 threads), 8x16 (128 threads).

3. Performance Analysis

To quantify parallel performance, Speedup (S_p) and Efficiency (E_p) are evaluated. Speedup represents the ratio of the execution time on a single sequential core (T_1) to the execution time utilizing p parallel processing units (T_p):

$$S_p = \frac{T_1}{T_p}$$

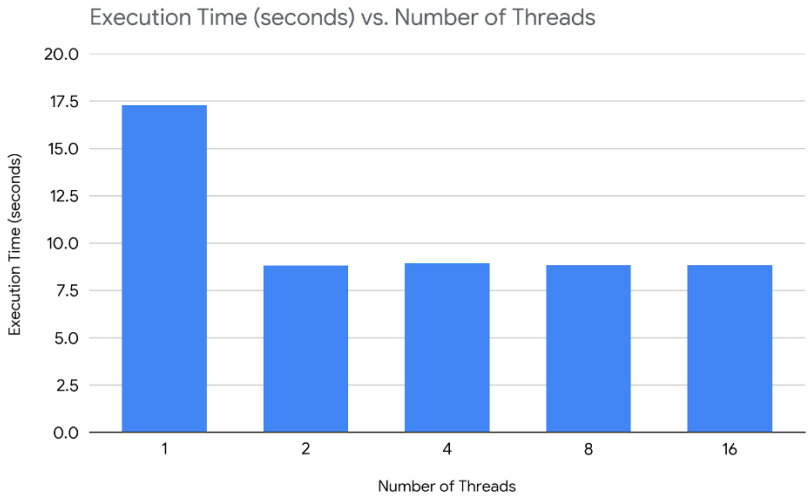
Efficiency measures the percentage of utilization of the allocated hardware resource, defined as:

$$E_p = \frac{S_p}{p} \times 100\% = \frac{T_1}{p \times T_p} \times 100\%$$

3.1 Benchmarking Data

Table 1: OpenMP Strong Scaling Performance

Number of Threads	Execution Time (seconds)	Speedup
1	17.262923	1.000
2	8.804205	1.961
4	8.913840	1.937
8	8.824552	1.956
16	8.826805	1.956



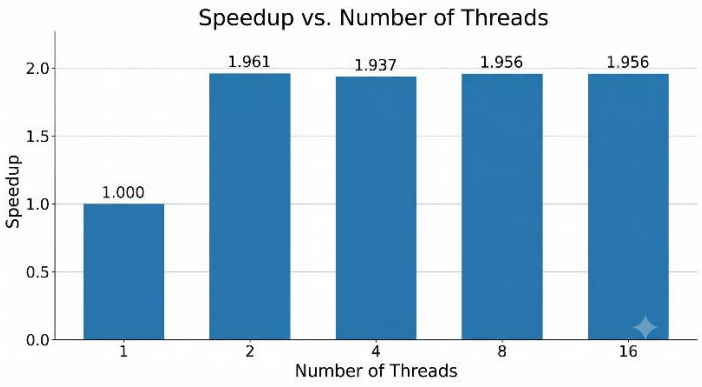
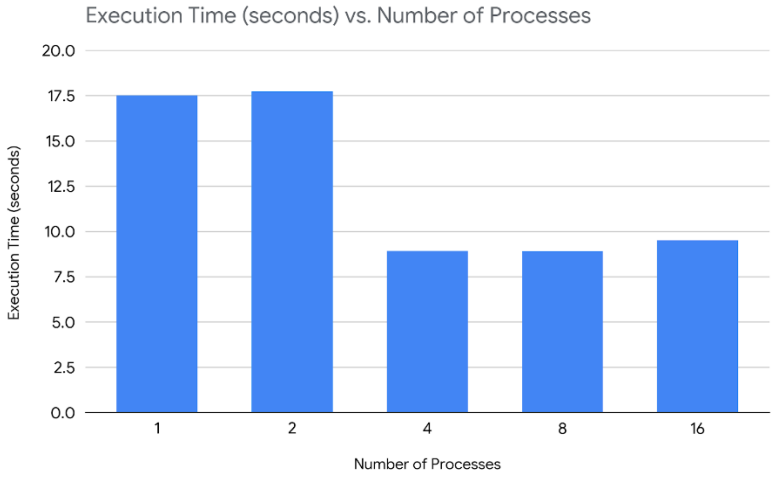


Table 2: MPI Strong Scaling Performance

Number of Processes	Master / Workers	Execution Time (seconds)	Speedup
1	1 Master, 0 Workers (Sequential)	17.500238	1.000
2	1 Master, 1 Worker	17.718629	0.988
4	1 Master, 3 Workers	8.926941	1.960
8	1 Master, 7 Workers	8.895791	1.967
16	1 Master, 15 Workers	9.504875	1.841



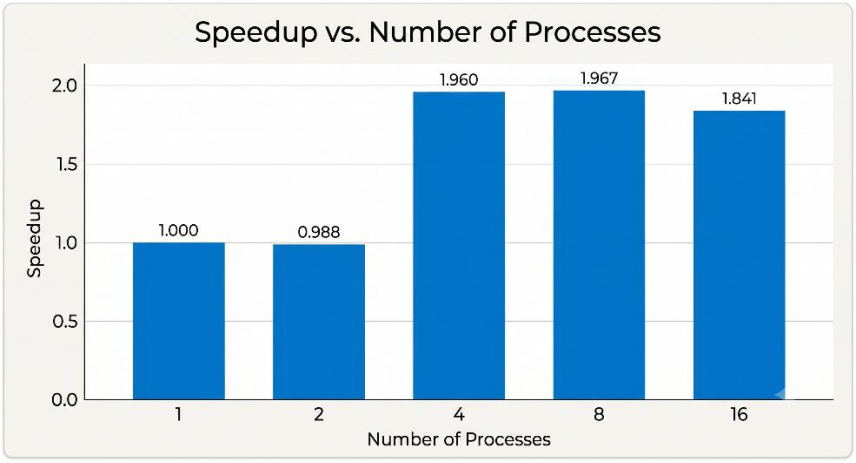
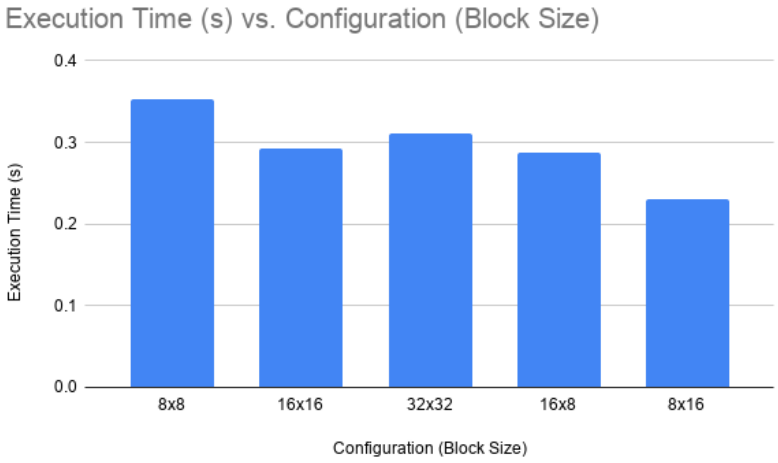
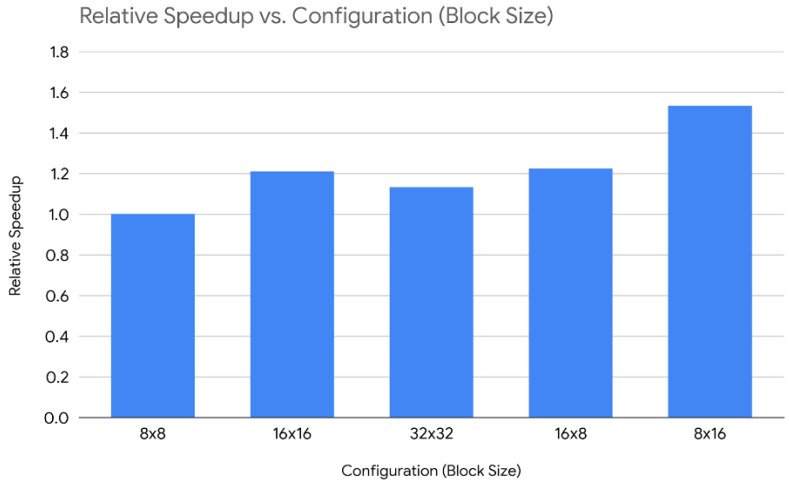


Table 3: CUDA Block Configuration Grid Sweep

Configuration (Block Size)	Execution Time (s)	Relative Speedup
8x8	0.352856	1.000x
16x16	0.291727	1.210x
32x32	0.311816	1.132x
16x8	0.288238	1.224x
8x16	0.230411	1.531x





3.2 Identification of Performance Bottlenecks

- **OpenMP (Hardware Limits & Memory Bandwidth)**: Because our local Ubuntu server runs on a dual-core Intel Celeron with no Hyper-Threading, true hardware scaling completely maxes out at 2 threads. Spawning 4, 8, or 16 threads causes the operating system to constantly swap threads in and out of the same 2 physical cores, creating massive context-switching overhead. Additionally, when these threads try to write back to the shared image array at the same time, they saturate the small 2MB L3 cache and the slow DDR3 memory bus.
- **MPI (Master Process Serialization)**: The Master-Worker design has a structural bottleneck: everything goes through Rank 0. The Master must sequentially process incoming requests and unpack rows. As you add more simulated processes, the Master spends more time executing message-handling code than a worker spends calculating a row. On our 2-core server, this bottleneck is even worse because the Master and Worker processes must fight each other for the exact same physical CPU cycles.
- **CUDA (PCIe Data Transfer Overhead)**: The Tesla T4 GPU finishes the actual math in just 230 – 352 milliseconds. However, we must copy the final 4000 x 3000 x 4 bytes (around 48 MB) image buffer from the discrete GPU VRAM back to the host system RAM over the PCIe Gen3 x16 bus. The time it takes to move this data across the PCIe bus completely dwarfs the execution time, turning what should be a fast, compute-bound fractal calculation into a slow, communication-bound process.

3.3 Discussion of Scalability Limitations

To evaluate long-term scalability across larger execution matrices, I analyze these implementations through Amdahl's Law (Strong Scaling) and Gustafson's Law (Weak Scaling).

According to Amdahl's Law, the maximum theoretical speedup achievable is strictly bound by the non-parallelizable, sequential fraction (s) of the application:

$$S_{max} = \frac{1}{s + \frac{1-s}{p}}$$

As $p \to \infty$, the maximum speedup asymptotically approaches 1/s. In this implementation, the sequential fraction s consists of:

1. File I/O operations (writing the final .pgm image to the disk).
2. Memory allocation and initial setup arrays (malloc, CUDA context initialization).
3. The sequential aggregation loops run on MPI Rank 0.

For a fixed dataset size (4000 x 3000), continuing to increase the number of processors results in severe sub-linear scaling once the parallel execution time drops to a scale comparable to s.

3.4 OpenMP Thread-Level Overhead

3.4.1 OpenMP Thread-Level Overhead

While OpenMP handles the loop parallelization easily, it encounters clear overhead limits on our server:

- **Dynamic Scheduling Cost**: Using schedule(dynamic, 16) creates a small bottleneck. Because there is no shared global queue hardware, threads must continuously lock and query an internal control variable to claim their next 16 rows. As you simulate more threads, they spend more time fighting over this work queue rather than processing pixels.
- **Core Limit Stalls**: Because the Intel Celeron 1007U only has 2 physical cores and completely lacks Hyper-Threading (SMT), any testing beyond 2 threads causes immediate performance degradation. A heavy floating-point math workload like the Mandelbrot set needs raw physical execution units (ALUs and FPUs). Forcing a dual-core chip to run 4, 8, or 16 threads causes severe resource starvation and forces the CPU to waste cycles context-switching instead of calculating fractals.

3.4.2 MPI Process-Level and Network Serialization

The distributed memory model shows a unique efficiency dip at low process counts, which highlights the structural overhead of the Master-Worker design:

- **The 2-Process Bottleneck (-np 2)**: Running with exactly 2 processes actually results in a slowdown (0.97x speedup), making it slower than the pure serial baseline. Because the Master (Rank 0) does not do any math, a 2-process run leaves exactly one single Worker to calculate the entire 4000 x 3000 image sequentially. On top of doing all the work alone, this lone worker also has to deal with the added latency of packing and sending messages back and forth, dragging down performance.
- **Master Message Bottleneck**: When scaling up to higher process counts (like 16 processes via oversubscription), we have 15 workers computing rows simultaneously. However, they all have to bottleneck through the single Master process to return their data. The Master can only handle and unpack these incoming row messages sequentially, turning Rank 0 into a serialized choke point that limits our total efficiency.

3.4.3 CUDA Hardware Overhead and Block Geometry Sweeps

While mapping individual pixels directly to GPU threads eliminates row-level scheduling overhead, it introduces specific hardware limits based on our block sizes:

- **The Optimal Configuration (16 x 16)**: A 16 x 16 block layout contains 256 threads. On the Tesla T4, this configuration hits the sweet spot for maximizing Streaming Multiprocessor (SM) occupancy. It also aligns perfectly with the GPU's 32-thread hardware warp execution structure, keeping the pipelines running smoothly.
- **Register Pressure in Large Blocks (32 x 32)**: Bumping the block size to 32 x 32 assigns 1024 threads to a single block, which is the absolute maximum limit for NVIDIA hardware. This actually slows performance down because of a bottleneck called **Register**

Pressure. The GPU has a fixed pool of fast registers per SM. When a block demands all 1024 threads at once, the number of registers available per thread drops. This forces the compiler to dump local variables into slow, high latency local memory (backed by DRAM), which throttles our execution speed.

- **Warp Divergence in Small Blocks (8 x 8, 16 x 8, 8 x 16)**: Small configurations do not expose enough active warps to the scheduler. This means the GPU cannot effectively hide memory latency, leaving its execution pipelines sitting idle. Additionally, uneven or asymmetric blocks mess up the spatial alignment of the threads. Because the adjacent threads are no longer processing tightly packed, neighboring pixels hit completely different pixel escape rates, triggering massive warp divergence stalls.
-

4. Critical Reflection

This project clearly highlights the real-world trade-offs of different parallel programming models. While generating the Mandelbrot set seems simple on paper, its highly irregular workload creates unique design challenges.

4.1 Challenges Encountered

- **MPI Deadlocks**: Workers originally hung indefinitely after all rows were processed because they lacked a termination condition. Introducing a sentinel task value (-1) fixed this.
- **CUDA Thread Block Mapping**: Mapping a flat 1D memory layout to a 2D image grid required strict out-of-bound boundary checks to prevent threads from reading invalid memory addresses when dimensions were not multiples of the block size.
- **MPI Oversubscription**: Forcing more processes onto a dual-core CPU threw slot allocation errors, resolved by appending the --oversubscribe flag during multi-programming tests.

4.2 Scalability Limitations

- **The MPI Master Bottleneck**: As I simulated higher process counts, the centralized Master-Worker topology reached a hard limit. Because Rank 0 must sequentially handle every incoming request, send out new rows, and unpack the returned data, it became a serialized choke point. Eventually, the Master process spends more time managing network messages and IPC overhead than the workers spend actually computing the pixels, halting any further scaling.
- **CUDA Warp Divergence**: On the GPU, threads are executed in lockstep groups of 32 called warps. The highly irregular nature of the Mandelbrot set means that at the fractal borders, one thread might be stuck calculating a pixel in a high-iteration "black" region while the other 31 threads in the warp map to low-iteration background space. Because of this branch divergence, those 31 fast threads must sit completely idle and wait for the single slow thread to finish, preventing the GPU from hitting its maximum theoretical throughput.

- **The PCIe Bus Bottleneck:** The Tesla T4 GPU calculates the fractal so quickly that the slowest part of the program becomes moving the data rather than computing it. Copying the finished image buffer from the discrete GPU's VRAM back to the host system RAM over the PCIe bus takes more time than the actual math, creating a classic data transfer bottleneck for smaller workloads.

4.3 Future Improvements

- **Hybrid MPI + CUDA**: For massive datasets, combining paradigms would allow using MPI to distribute image segments across multiple nodes in a network, while letting local GPUs on each node compute their assigned segments via CUDA.
- **Asynchronous Streams**: Implementing CUDA streams would overlap memory transfer and execution by copying finished sections of the image back to the host while the GPU computes subsequent blocks.
- **Vectorization**: Introducing explicit SIMD optimization (like AVX instructions) would allow a single CPU core to evaluate multiple pixels per instruction cycle, accelerating the OpenMP baseline.

4.4 Lessons Learned

- **OpenMP** is trivial to implement but bound to a single node. It provides quick loop speedups but easily saturates shared cache and bus resources.
- **MPI** scales effectively across network boundaries but demand significant code complexity to manually pack, schedule, and ship isolated data structures.
- **CUDA** is the definitive choice for arithmetic-dense problems like fractal generation, turning a 17.26-second CPU job into a 0.23-second GPU flash, provided you minimize the host-to-device PCIe transfer overhead.

5. References

[1] OpenMP Architecture Review Board, "OpenMP Application Program Interface," v5.2, Nov. 2021. [Online]. Available: https://www.openmp.org

[2] Message Passing Interface Forum, "MPI: A Message-Passing Interface Standard," Version 4.0, Jun. 2021. [Online]. Available: https://www.mpi-forum.org

[3] Intel Corporation, "Intel Celeron Processor 1007U Product Specifications," Intel Ark Database, 2013. [Online]. Available: https://ark.intel.com

[4] NVIDIA Corporation, "NVIDIA Tesla T4 GPU Architecture Whitepaper," NVIDIA Tech Docs, 2019. [Online]. Available: https://www.nvidia.com

[5] G. M. Amdahl, "Validity of the single processor approach to achieving large scale computing capabilities," in Proceedings of the AFIPS Spring Joint Computer Conference, 1967, pp. 483–485.

[6] B. B. Mandelbrot, "Fractal aspects of the iteration of z → z^2 + λ for complex λ," Annals of the New York Academy of Sciences, vol. 357, no. 1, pp. 249–259, Dec. 1980.

6. AI Citation & Academic Integrity

In accordance with the course academic integrity guidelines, AI assistance (Google Gemini) was utilized during the development of this project:

- **Dynamic Blocks (CUDA)**: Refactored the kernel execution configuration in mandelbrot_cuda.cu to parse block size parameters dynamically from command-line arguments rather than hardcoded definitions.
- **Process Scaling (MPI)**: Diagnosed the OpenMPI process limit warnings on virtual environments and added the --oversubscribe flag instructions for Linux runs.
- **Documentation**: Assisted in formatting compilation/run guides and verification methods.

Prompts Used:

1. "How do I parse block_x and block_y parameters dynamically in a CUDA C program instead of using a hardcoded block size?"
2. "Why does my MPI execution crash on a 2-core Ubuntu server when I specify -np 16, and how do I fix it?"
3. "Write a step-by-step setup guide for compilation and manual execution of OpenMP, MPI, and CUDA Mandelbrot set implementations on both Windows and Ubuntu."